ROW MAJOR

In this method the elements are stored row wise.

i.e. n elements of first row are stored in first n locations, n elements of second row are stored in next n locations and so on.

A[0][0]	A[0][1]	A[0][2]	A[0][3]
A[1][0]	A[1][1]	A[1][2]	A[1][3]
A[2][0]	A[2][1]	A[2][2]	A[2][3]

3*4

Element	Address
A[0][0]	200
A[0][1]	201
A[0][2]	202
A[0][3]	203
A[1][0]	204
A[1][1]	205
A[1][2]	206
A[1][3]	207
A[2][0]	208
A[2][1]	209
A[2][2]	210
A[2][3]	211

COLUMN MAJOR

In this method the elements are stored column wise.

i.e. m elements of first column are stored in first m locations, m elements of second column are stored in next m locations and so on.

A[0][0]	A[0][1]	A[0][2]	A[0][3]
A[1][0]	A[1][1]	A[1][2]	A[1][3]
A[2][0]	A[2][1]	A[2][2]	A[2][3]

Element	Address
A[0][0]	200
A[1][0]	201
A[2][0]	202
A[0][1]	203
A[1][1]	204
A[2][1]	205
A[0][2]	206
A[1][2]	207
A[2][2]	208
A[0][3]	209
A[1][3]	210
A[2][3]	211

ADDRESS CALCULATION

Row Major System:

Address of A [I][J] = B.A + [(I – Lr)* N + (J – Lc)]*Size

Column Major System:

Address of A [I][J] = B.A + [(I – Lr) + (J – Lc)*M]*Size

ADDRESS CALCULATION

B.A = Base address
I = Row subscript of element whose address is to be found
J = Column subscript of element whose address is to be found
Size = Storage Size of one element stored in the array (in byte)
Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)
Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)
M = Number of row of the given matrix
N = Number of column of the given matrix

Usually number of rows and column of a matrix are given like A[10][15], A[5][2] but if it is given as A[Lr.....Ur][Lc.....Uc]

So in this case number of rows and columns will be calculated as

rows(M)= (Ur–Lr)+1

column(N)=(Uc–Lc)+1

2D Array Example

Consider a 3x 4 array A. Suppose the Base(A) = 200 and w =1. Suppose the programming store 2D array.
Compute LOC(A[1,2])
Row-major.

- LOC(A[J,K]) = Base(A) + w[n(J-1) + (K-1)]

LOC(A[1,2]) = 200 + 1[4(1-0) + (2 -0)]= 206

Coloumn-major.
LOC(A[1,2]) = 200 + 1[(1-0) +3(2-0)]= 207

A **2-dimensional array** such as

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$$

can be represented in computer memory in several ways. The key idea is that **memory one-dimensional**, so a 2D array has to be mapped onto a linear sequence of memory

1. Row-major representation

Store elements **row by row**:

$$a_{00}, a_{01}, a_{02}, a_{10}, a_{11}, a_{12}, a_{20}, a_{21}, a_{22}$$

Memory looks like:

```
Address →
+------+------+------+------+------+------+------+------+------+
| a00  | a01  | a02  | a10  | a11  | a12  | a20  | a21  | a22  |
+------+------+------+------+------+------+------+------+------+
```

2. Column-major representation

Store elements **column by column**:

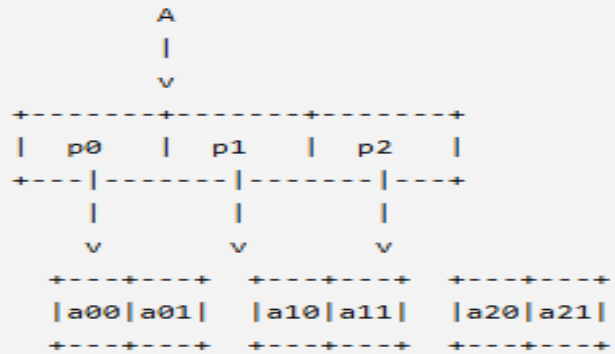
$$a_{00}, a_{10}, a_{20}, a_{01}, a_{11}, a_{21}, a_{02}, a_{12}, a_{22}$$

Memory:

```
Address →
+------+------+------+------+------+------+------+------+------+
| a00  | a10  | a20  | a01  | a11  | a21  | a02  | a12  | a22  |
+------+------+------+------+------+------+------+------+------+
```

3. Array of pointers

Another approach is to store an array of pointers, where each pointer points to a separate row:



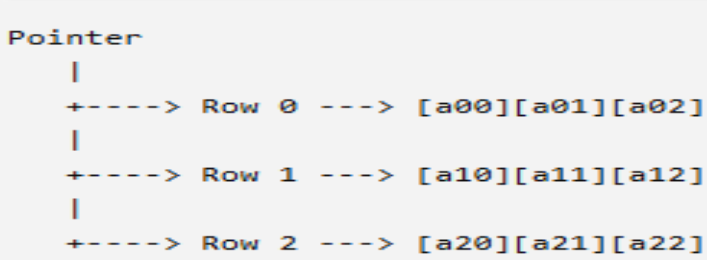
The rows **do not necessarily need to be contiguous** in memory.

This is useful when rows have different lengths, giving a **jagged array**.

4. Dynamically allocated 2D array

We can also allocate memory dynamically.

Conceptually:



This gives flexibility when the dimensions are known only at runtime.

Polynomial Manipulation

- A polynomial is a mathematical expression consisting of a sum of terms, each term including a variable or variables raised to a power and multiplied by a coefficient. The simplest polynomials have one variable.

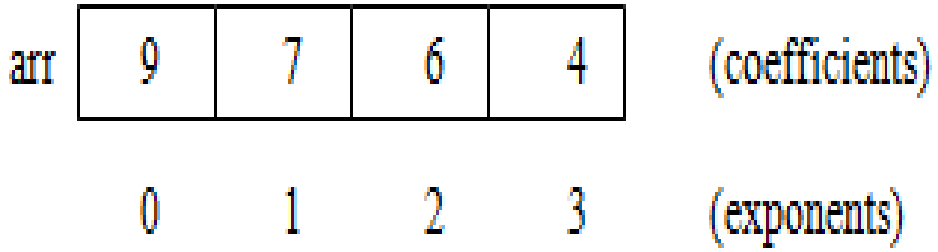
- **Representation of a Polynomial:** A polynomial is an expression that contains more than two terms. A term is made up of coefficient and exponent. An example of polynomial is

  - $P(x) = 4x^3+6x^2+7x+9$

$$f(x) = \sum_{i=0}^{n} a_i x^i$$

A polynomial thus may be represented using arrays or linked lists.

- Array representation assumes that the exponents of the given expression are arranged from 0 to the highest value (degree), which is represented by the subscript of the array beginning with 0.
- The coefficients of the respective exponent are placed at an appropriate index in the array.
- The array representation for the above polynomial expression is given below:



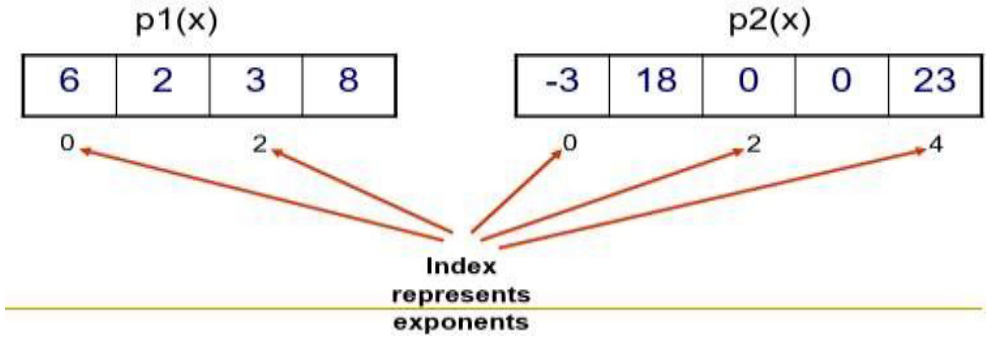
Here are the most common operations on a polynomial:

- Add & Subtract
- Multiply
- Differentiate
- Integrate
- etc…

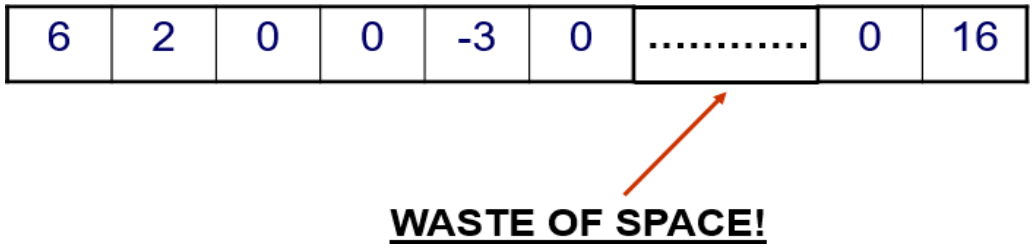
There are different ways of implementing the polynomial ADT:

- Array (not recommended)
- Linked List (preferred and recommended)

  Array Implementation:
  - $p1(x) = 8x^3 + 3x^2 + 2x + 6$
  - $p2(x) = 23x^4 + 18x - 3$



- This is why arrays aren't good to represent polynomials:
- $p3(x) = 16x^{21} - 3x^5 + 2x + 6$



Consider:

$$P(x, y) = 3x^2y + 5xy^2 + 7x^2 + 4y + 6$$

So the polynomial can be represented by:

$$A = \begin{bmatrix} 6 & 4 & 0 \\ 0 & 0 & 5 \\ 7 & 3 & 0 \end{bmatrix}$$

Suppose:

$$P(x, y) = 2x^3 + 4x^2y^2 - 7xy + 5y^3 + 8$$

For example:

- $A[0][0] = 6 \rightarrow 6x$
- $A[0][1] = 4 \rightarrow 4x^0y^1 = 4y$
- $A[1][2] = 5 \rightarrow 5x^1y^2 = 5xy^2$
- $A[2][0] = 7 \rightarrow 7x^2y^0 = 7x^2$
- $A[2][1] = 3 \rightarrow 3x^2y^1 = 3x^2y$

Suppose:

$$P(x, y) = 2x^3 + 4x^2y^2 - 7xy + 5y^3 + 8$$

The coefficient array would be:

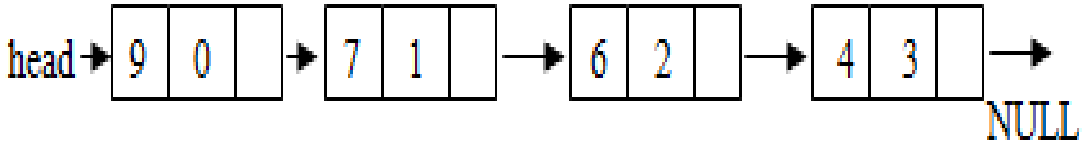
$$A = \begin{bmatrix} 8 & 0 & 0 & 5 \\ 0 & -7 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

- Advantages of using an Array:
  - only good for non-sparse polynomials.
  - ease of storage and retrieval.
- Disadvantages of using an Array:
  - have to allocate array size ahead of time.
  - huge array size required for sparse polynomials. Waste of space and runtime.

- A polynomial may also be represented using a linked list. A structure may be defined such that it contains two parts- one is the coefficient and second is the corresponding exponent. The structure definition may be given as shown below:

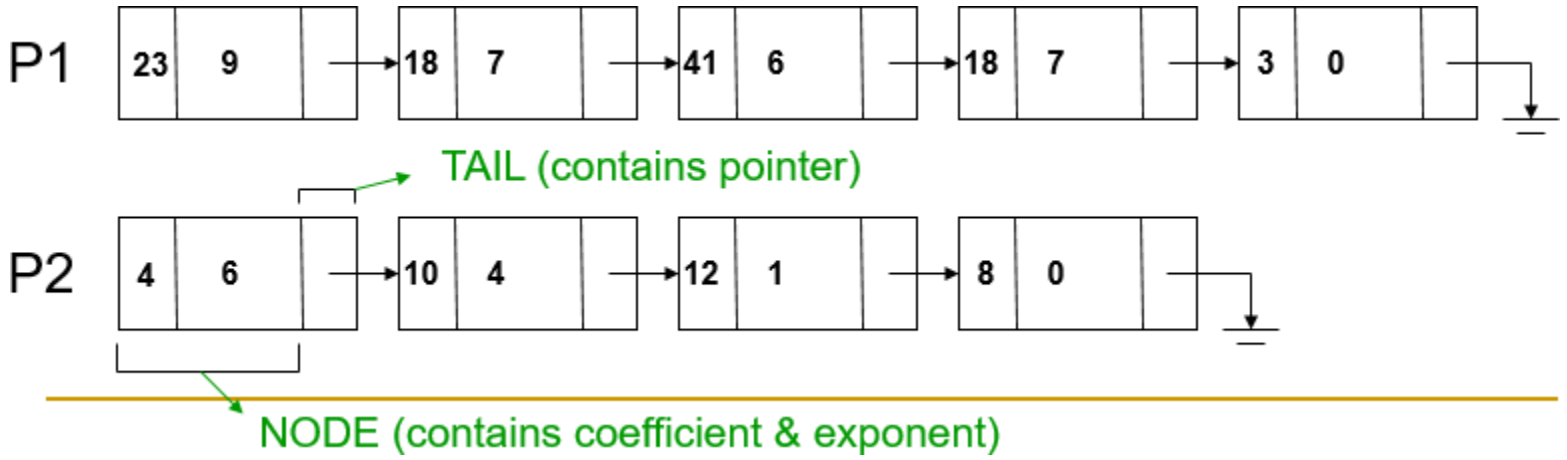
```
struct polynomial
{
int coefficient;
int exponent;
struct polynomial *next;
};
```

- Thus the above polynomial may be represented using linked list as shown below:



Linked list Implementation:

- $p1(x) = 23x^9 + 18x^7 + 41x^6 + 163x^4 + 3$
- $p2(x) = 4x^6 + 10x^4 + 12x + 8$



Representation

```
struct polynode {
      int coef;
      int exp;
      struct polynode * next;
  };

typedef struct polynode *polyptr;
```



- Advantages of using a Linked list:
  - save space (don't have to worry about sparse polynomials) and easy to maintain
  - don't need to allocate list size and can declare nodes (terms) only as needed
- Disadvantages of using a Linked list :
  - can't go backwards through the list
  - can't jump to the beginning of the list from the end.

Addition of two Polynomials:

- For adding two polynomials using arrays is straightforward method, since both the arrays may be added up element wise beginning from 0 to n-1, resulting in addition of two polynomials.
- Addition of two polynomials using linked list requires comparing the exponents, and wherever the exponents are found to be same, the coefficients are added up.
- For terms with different exponents, the complete term is simply added to the result thereby making it a part of addition result.

$$\begin{array}{r} x^5 + 3x^3 + 4 \\ +\ 6x^6 + \quad +4x^3 \\ \hline 6x^6 + x^5 + 7x^3 + 4 \end{array}$$

Adding polynomials using a Linked list representation: (storing the result in p3)

To do this, we have to break the process down to cases:

Case 1: exponent of p1 > exponent of p2

- Copy node of p1 to end of p3.
- [go to next node]

Case 2: exponent of p1 < exponent of p2

- Copy node of p2 to end of p3.
- [go to next node]

Case 3: exponent of p1 = exponent of p2

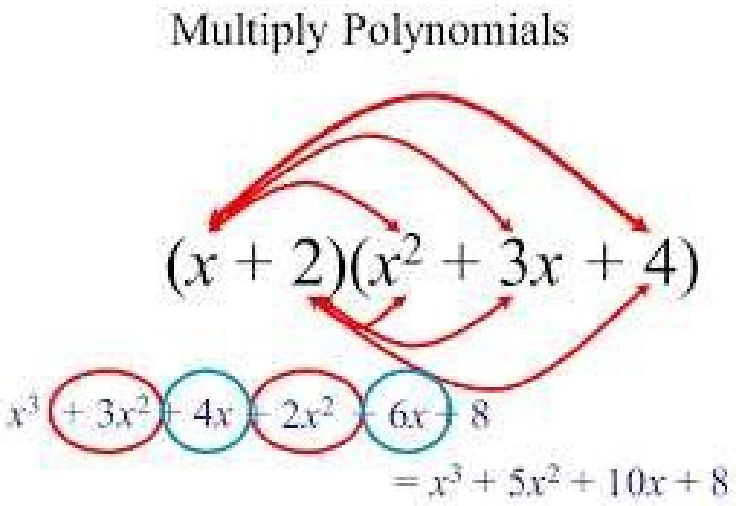
- Create a new node in p3 with the same exponent and with the sum of the coefficients of p1 and p2.

CODING

Multiplication of two Polynomials:

Multiplication of two polynomials however requires manipulation of each node such that the exponents are added up and the coefficients are multiplied.

After each term of first polynomial is operated upon with each term of the second polynomial, then the result has to be added up by comparing the exponents and adding the coefficients for similar exponents and including terms as such with dissimilar exponents in the result.



All Operations (Insertion, Deletion, Merge, Traversal)

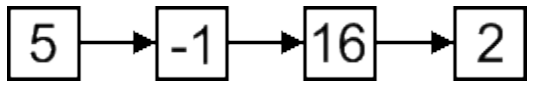
- **Insertion**
- **Deletion**
- **Merge two sorted linked lists**
- **Traversal**
  - Assume, that we have a list with some nodes. Traversal is the very basic operation, which presents as a part in almost every operation on a singly-linked list.
  - For instance, algorithm may traverse a singly-linked list to find a value, find a position for insertion, etc. For a singly-linked list, only forward direction traversal is possible.

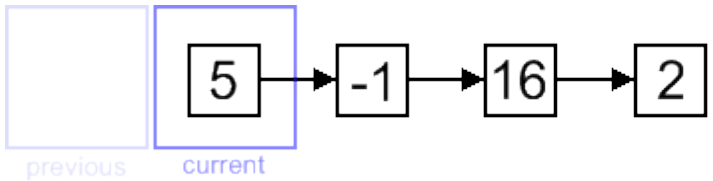
Traversal algorithm

- Beginning from the head,
- check, if the end of a list hasn't been reached yet;
- do some actions with the current node, which is specific for particular algorithm;
- current node becomes previous and next node becomes current. Go to the step 1.

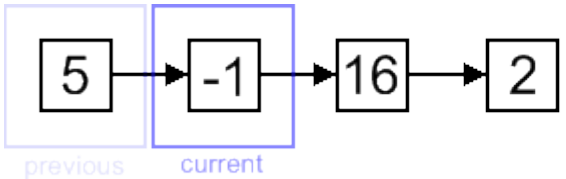
Example

- As for example, let us see an example of summing up values in a singly-linked list.

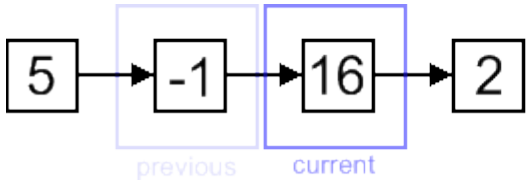




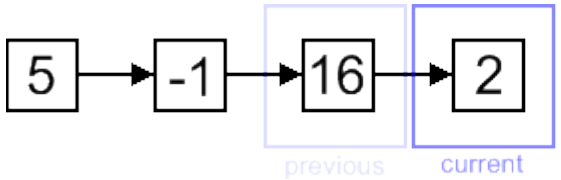
initial state
sum = 0



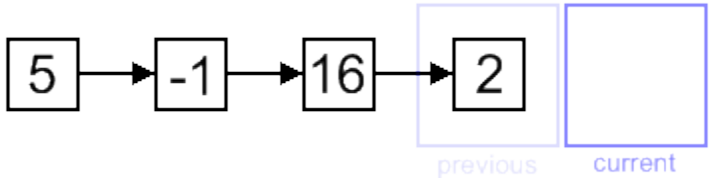
step 1
sum = 5



step 2
sum = 4



step 3
sum = 20



final state
sum = 22